

SQLite Commands in Django (Summary)

1. Migrations (Create / Update Database Schema):

2. `python manage.py makemigrations`: Generate migration files from changes in models.
3. `python manage.py migrate`: Apply migrations to update the SQLite database.

4. Inspect Database Schema:

5. `python manage.py showmigrations`: Show list of migrations and their status.
6. `python manage.py sqlmigrate app_name migration_number`: Show raw SQL for a migration.

7. Run SQL Commands / Queries:

8. Use Django's ORM methods in code:

- `Model.objects.all()`
- `Model.objects.filter(...)`

9. Or access DB shell:

- `python manage.py dbshell`: Opens an interactive SQLite shell if installed.

10. Reset / Recreate Database:

11. Delete `db.sqlite3` file manually, then rerun:

- `python manage.py migrate`

12. Dump / Load Data:

13. `python manage.py dumpdata > data.json`: Export data to JSON.
14. `python manage.py loaddata data.json`: Import data from JSON.

15. Check Database Configuration:

16. In `settings.py`, confirm:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',
```

```
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

17. Raw SQL Execution in Django:

18. Using Django connection object:

```
from django.db import connection  
  
with connection.cursor() as cursor:  
    cursor.execute("SELECT * FROM myapp_mymodel")  
    rows = cursor.fetchall()
```

Note: SQLite is the default database for Django projects and great for local/testing environments.

`makemigrations` creates migration files (Python scripts) inside your app's `migrations` directory.

These files (e.g., `0001_initial.py`, `0002_auto_20250630_1234.py`) contain Python code describing the schema changes (like adding fields, tables, etc.).

When you run `migrate`, Django reads those migration files and applies the described changes to the actual database (like updating `db.sqlite3`).